Digital Art Layer Manager using a Circular Doubly Linked List (CDLL)

Data Structures Project Report

1. Abstract

Digital art tools such as Photoshop, Krita and Procreate organise artwork as a stack of layers. This report presents a Layer Manager built on a **Circular Doubly Linked List (CDLL)**. Every layer is a node holding its ID, name and visibility flag, and is linked to the layer on either side of it. Because the list is circular, the last node points back to the first, so the user can cycle from the top layer to the bottom layer (and back) seamlessly, which suits graphics loops. The report covers the node design, the algorithms for insertion, deletion and traversal, their complexity, flowcharts, the C++ implementation and sample output.

2. Problem Statement

The earlier version of the project stored layers in a `vector` (used as a stack) and used a `queue` as a garbage-collection buffer for inactive layers. That design has two weaknesses: inserting at the bottom or deleting from the middle needs O(n) element shifting, and the structure has no natural wraparound for loop-style navigation.

The modified problem replaces it with a **Doubly Linked List variation with structural refinements**, namely a **Circular Doubly Linked List**, where:

- insertion at the top is O(1) because the tail is always reachable as `head->prev`;
- deletion of any known node is O(1) pointer re-linking;
- traversal can start at any node and always terminates when it returns to the starting node;
- cycling from the top layer to the bottom layer needs no special case.

3. Objectives

1. Represent each canvas layer as a node of a CDLL.
2. Insert a new layer at the top (head) in constant time.
3. Delete any layer by ID while keeping the circular links intact.
4. Render (traverse) the whole canvas forward, stopping when the traversal returns to the head.

5. Toggle layer visibility and provide a menu-driven console interface.

4. Data Structure Design

Each node contains:

Field	Type	Purpose
layerID	Integer	Unique identifier of the layer
name	String	Friendly layer name
isVisible	Boolean	Visibility flag (true = shown, false = muted)
next	Pointer	Pointer to the next layer in the circle
prev	Pointer	Pointer to the previous layer in the circle

The list is accessed through a single pointer `head`, which marks the top layer. When the list is empty, `head = NULL`. For a non-empty list, the invariants are:

- `head->prev` is the tail (the last node);
- `tail->next` is `head`;
- for every node `x`: `x->next->prev == x` and `x->prev->next == x`;
- with a single node, `next` and `prev` both point to the node itself.

5. Algorithms

5.1 Initialization

Set `head = NULL`.

5.2 Insertion at Top (Head)

1. Create a new node N.
2. If `head` is NULL, point N's `next` and `prev` to N itself, then set `head = N`.
3. If `head` exists, find the last node: `tail = head->prev`.
4. Set `N->next = head` and `N->prev = tail`.
5. Update `tail->next = N` and `head->prev = N`.
6. Update `head = N`.

Because the tail is found through `head->prev`, no loop is needed, and the operation takes O(1) time.

5.3 Deletion of a Layer

1. If the list is empty, return.
2. Locate the target node T by searching from `head` until the ID matches or the traversal returns to `head`. If T is the only node, set `head = NULL`.
3. Otherwise set `T->prev->next = T->next`.
4. Set `T->next->prev = T->prev`.
5. If `T == head`, move head to `T->next`.
6. Deallocate the memory of T.

Once T is located, the unlinking itself is O(1). Locating T by ID costs O(n).

5.4 Forward Traversal (Render Canvas)

1. Start at `head`.
2. Loop, printing or processing the details of each node.
3. Move to `current->next`. Stop when `current == head` again.

A `do-while` loop is used so that the head is processed first, even though the stop condition compares against the head.

5.5 Supporting operations

- **Insertion at Bottom:** perform steps 1 to 5 of the top insertion but do not update `head`. The new node then sits just before the head in the circle, which is the bottom of the stack.
- **Toggle visibility:** locate the node by ID using the same circular search, then set `isVisible`.

6. Complexity Analysis

Operation	Time	Space
Insert at top	O(1)	O(1)
Insert at bottom	O(1)	O(1)
Delete a layer (search + unlink)	O(n) search, O(1) unlink	O(1)
Toggle visibility	O(n)	O(1)
Render canvas	O(n)	O(1)

The whole structure uses O(n) space, with two pointers of overhead per layer.

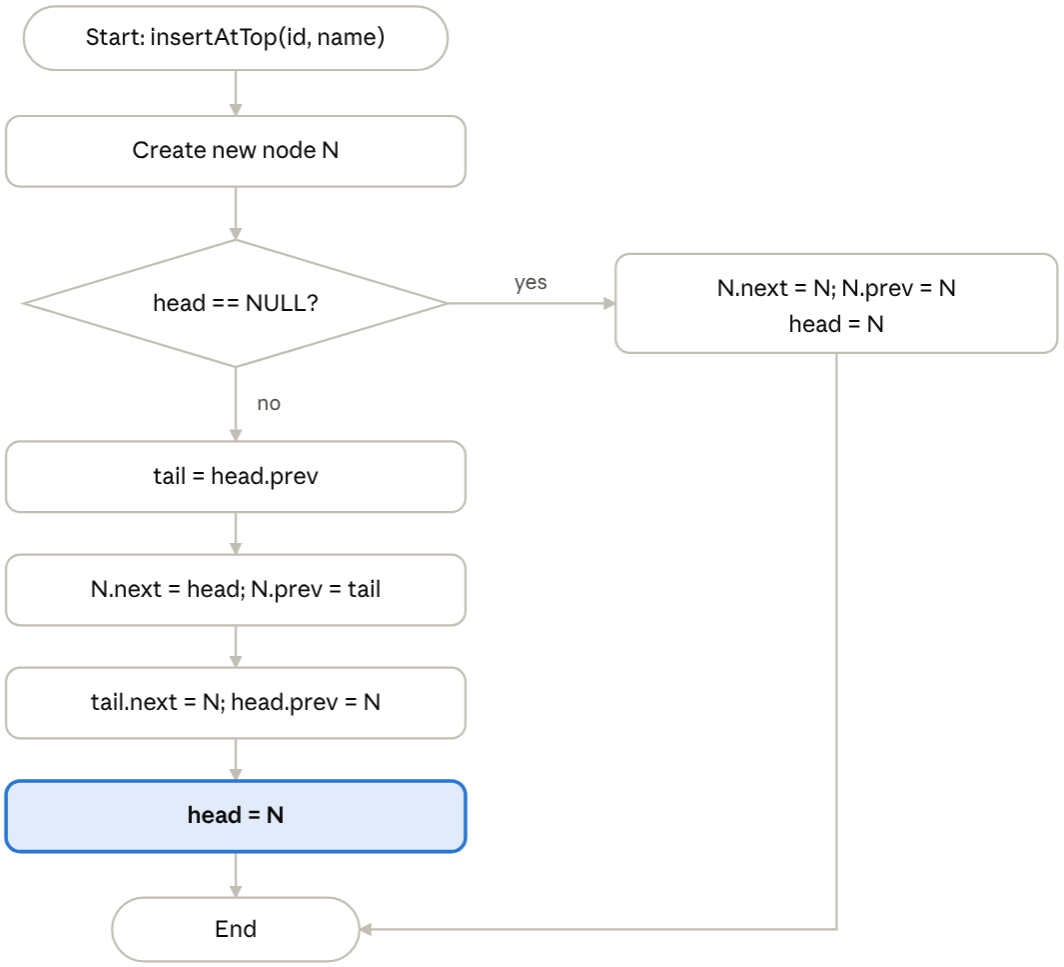
7. Comparison with the Earlier Design

Aspect	Vector stack + queue	Circular doubly linked list
Insert at bottom	O(n), shifts all elements	O(1)
Delete from the middle	O(n), rebuilds vector	O(1) once located
Wraparound from top to bottom	Needs special handling	Built in
Extra buffer for removed layers	Garbage queue	Not needed, node freed directly
Memory per layer	One pointer in the vector	Two extra pointers per node

8. Flowcharts

8.1 Insertion at Top

Insertion at the top takes constant time

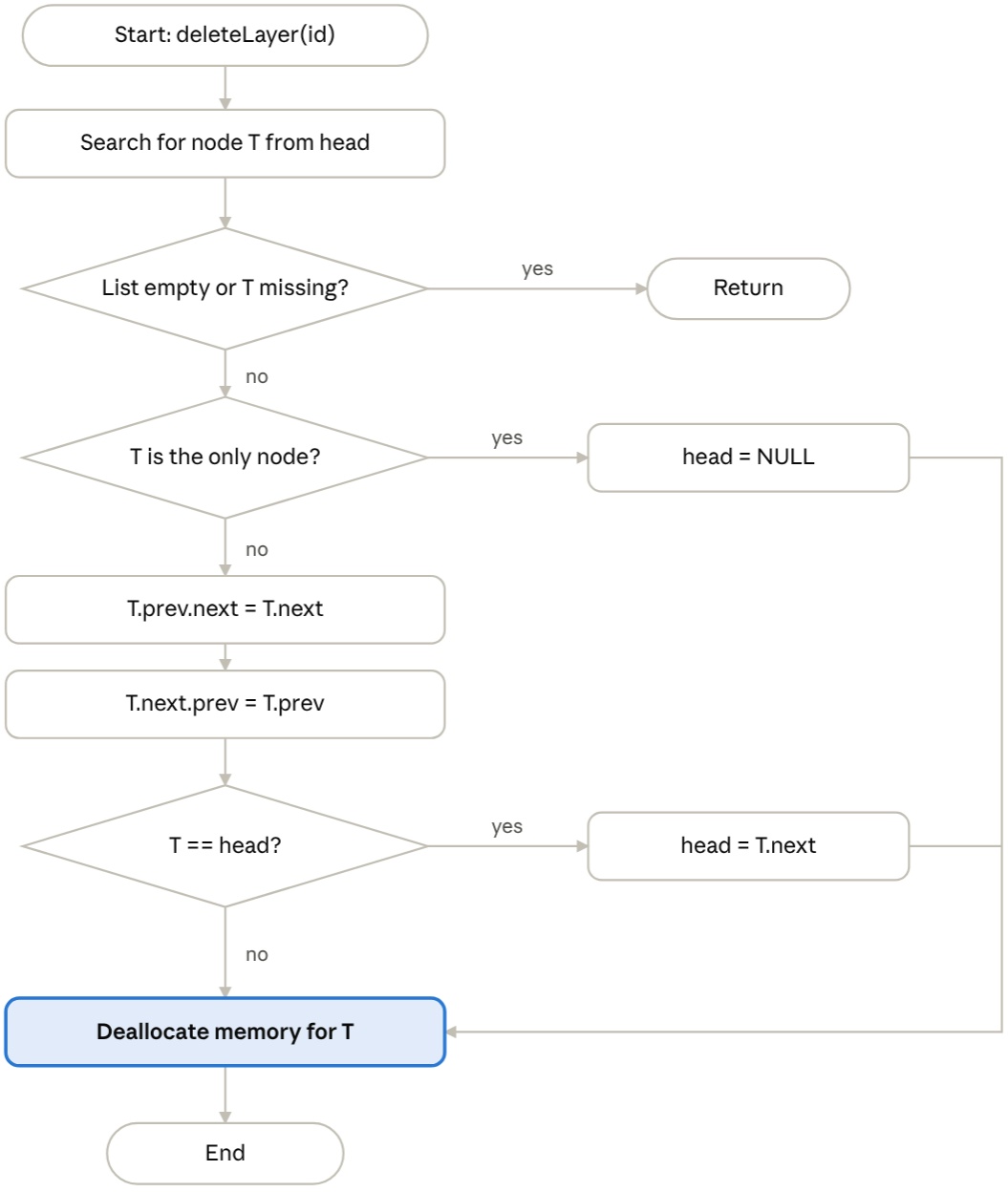


insertion at top · 1 decision

The tail is reached through `head->prev`, so both branches finish in a fixed number of steps.

8.2 Deletion of a Layer

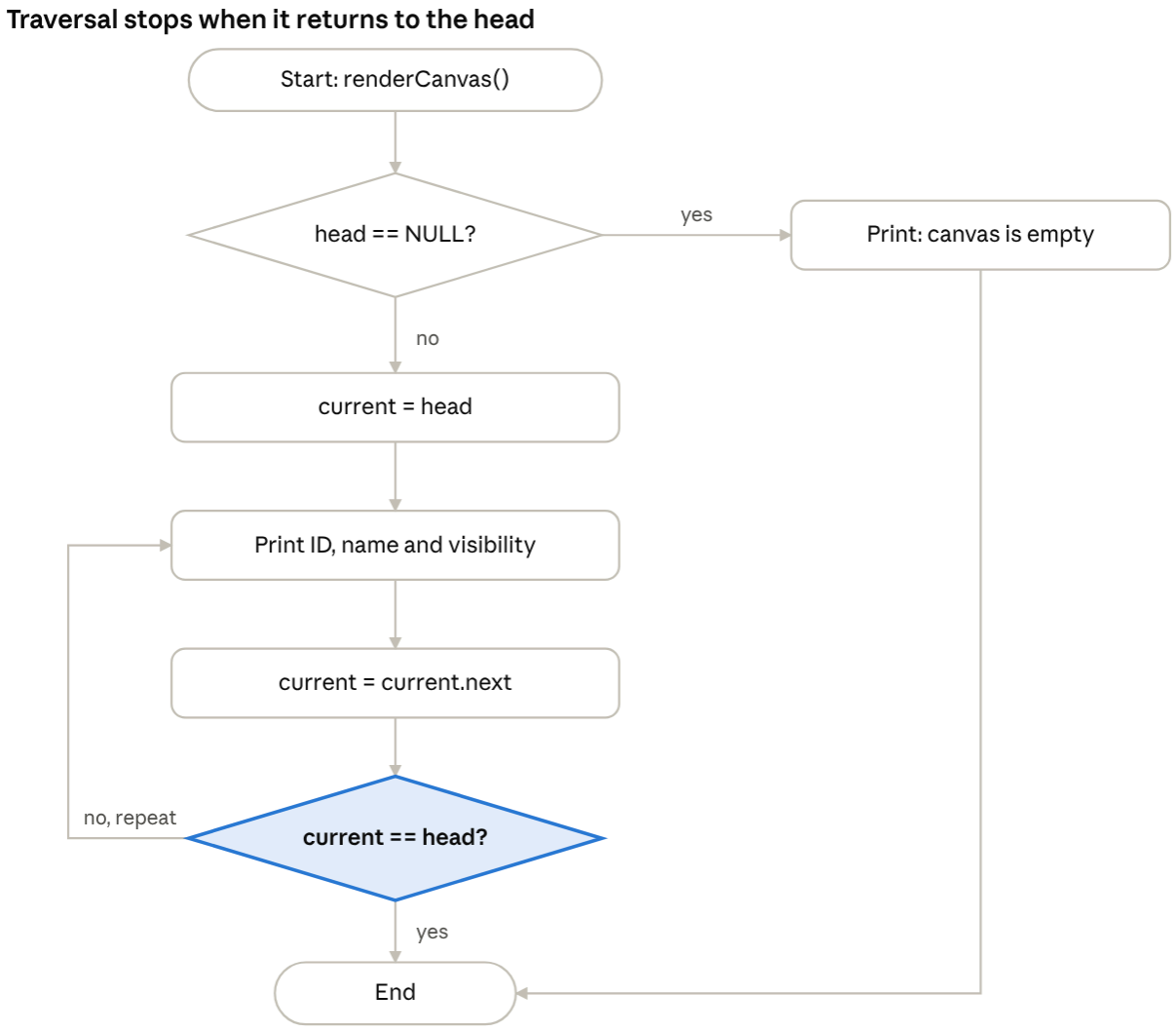
Deletion re-links two pointers, then frees the node



deletion of a layer · 3 decisions

A list with one node is emptied; otherwise the two neighbours are joined and the head moves only if the head itself was removed.

8.3 Forward Traversal (Render Canvas)



forward traversal · loop with 1 exit test

The loop returns to its first step until `current` wraps around to the head, which ends the render.